Design Document: SyncText

A CRDT-Based Collaborative Text Editor

CS69201 – Computing Lab

Sumit

25CS60R63

November 9, 2025

Contents

1 System Architecture

1.1 High-Level Design Overview

SyncText is a decentralized, multi-process, multi-threaded collaborative editing system. It simulates the real-time functionality of Google Docs without relying on a central server for synchronization logic.

Each user runs an independent `editor` process. The system's architecture is built on three core, lock-free principles as required by the project:

1. **Shared Memory Registry (for Discovery):** A lock-free shared memory segment acts as a central "phone book." When a user process starts, it registers its Process ID (PID) and message queue name using atomic operations. All other users can poll this registry to discover who is currently online.

2. **Message Queues (for Communication):** Each user process creates its own unique POSIX Message Queue. This queue acts as a personal "mailbox" for receiving updates. When a user makes a change, their process broadcasts it by sending an update message to the message queues of all other active users (discovered via shared memory).

3. **CRDT (for Synchronization):** Consistency is achieved using a Conflict-Free Replicated Data Type (CRDT) model. All concurrent updates are guaranteed to merge correctly and deterministically using a Last-Writer-Wins (LWW) strategy, which relies on timestamps and ‘user$_id$‘$tiebreakers.Thisentireprocessislock - free, asrequired.$

1.2 Major Components and Interaction

The system is modularized into several key components:

- **Registry (`registry.cpp`):** Manages all interactions with the shared memory segment. Handles atomic user registration and de-registration.
- **Messaging (`messaging.cpp`):** Manages all POSIX Message Queue operations. Handles creation of the user's own queue, broadcasting updates, and data serialization.
- **File Monitor (`file_monitor.cpp`):** Responsible for polling the local document using `stat()`, detecting changes, and running the ‘diff‘ algorithm to create ‘UpdateObject‘s.
- **CRDT (`crdt.cpp`):** The core synchronization logic. Implements conflict detection and the LWW resolution algorithm.
- **Display (`display.cpp`):** A simple UI component for clearing the terminal and rendering the document state and active user list.
- **Main (`main.cpp`):** The main driver. It initializes all components, launches the threads, and runs the main event loop.

1.3 High-Level Architecture Diagram

This diagram shows the inter-process communication flow. Each user process discovers all others via the central Shared Memory and then broadcasts updates directly to their peers' Message Queues.

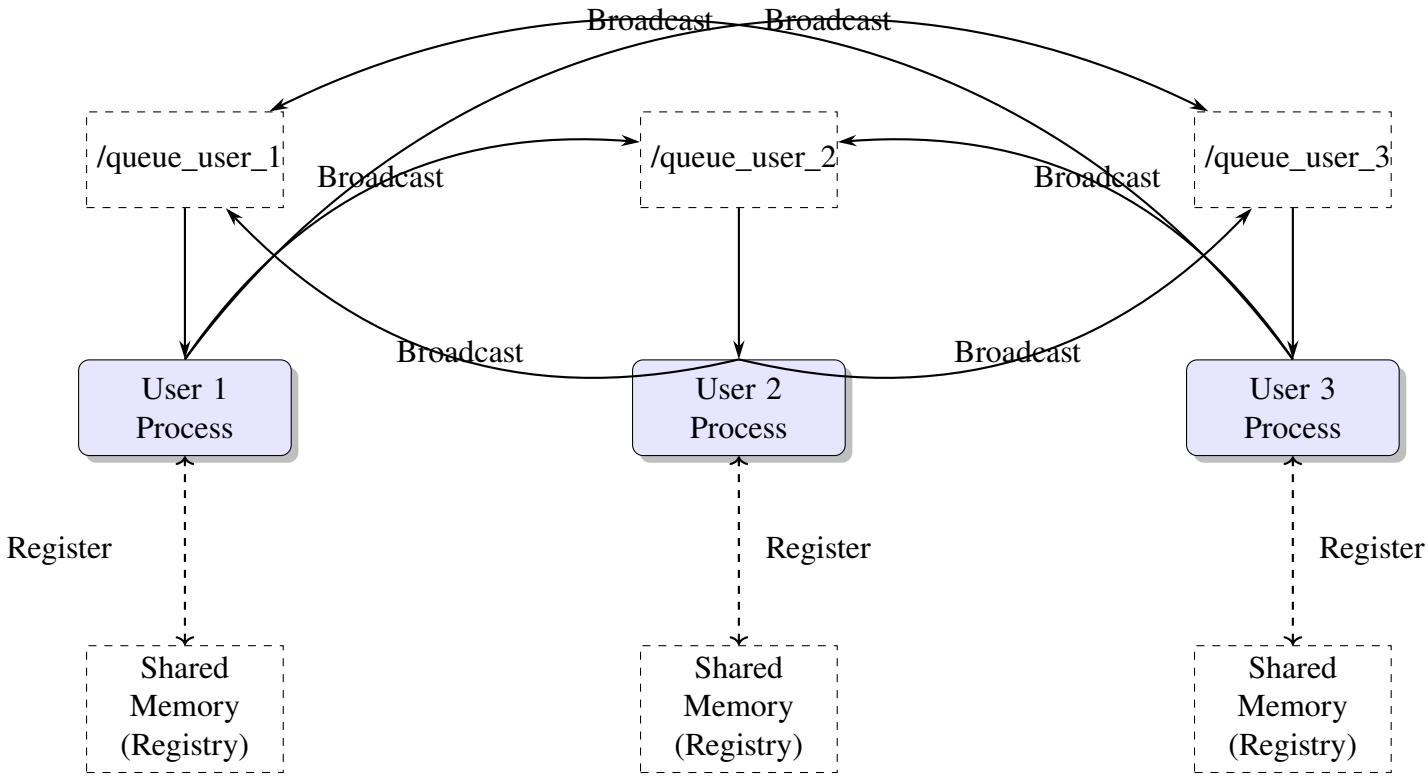

Figure 1: High-Level System Architecture. Discovery is centralized (via SHM), but communication is peer-to-peer (via MQs).

1.4 Internal Thread Architecture

This diagram in Figure 2 shows the internal architecture of a *single* `editor` process, as required by the project. The system is multi-threaded to handle local file monitoring and remote updates concurrently without locks.

1.5 Key Data Structures

- **UpdateObject (C++):** The primary internal struct. It uses `std::string` to hold `old_content` and `new_content`, making it flexible but variable-sized.
- **MQ_UpdateObject (C):** A fixed-size, C-style struct. It uses `char[]` arrays for content. This is the "on-the-wire" format required for POSIX Message Queues, which demand a constant message size.
- **SharedRegistry:** The struct mapped to shared memory. Its key component is `UserInfo users[MAX_USERS]`, where `UserInfo` contains an `std::atomic<pid_t>`. This atomic variable is the key to our lock-free registration.
- **SPSCQueue:** A template-based, lock-free, single-producer, single-consumer queue. This is the internal "conveyor belt" that passes received `MQ_UpdateObject`s from the Listener Thread to the Main Thread safely.

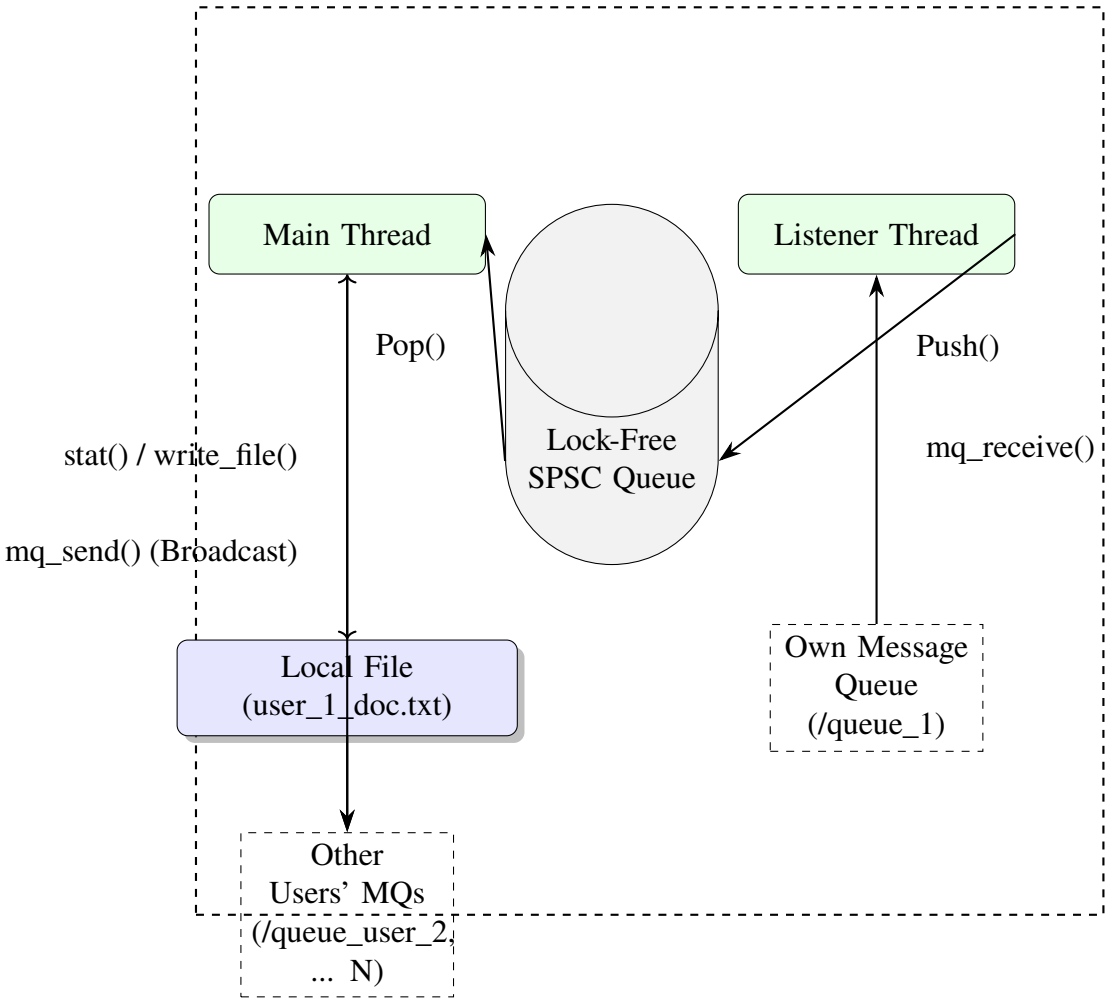

Figure 2: Internal Lock-Free Thread Architecture. The SPSC Queue is the only communication channel, eliminating the need for mutexes.

2 Implementation Details

2.1 Change Detection (File Monitoring)

This requirement is met by a polling mechanism in the Main Thread.

1. The thread stores the file's `last_mod_time` in a local variable.
2. In each loop, it calls `get_mod_time()`, which is a wrapper for the `stat()` system call, to get the file's current modification time.
3. If `new_mod_time > last_mod_time`, a change is detected.
4. The system then calls `diff_documents()`. This function compares the in-memory `local_content` vector with a new vector read from the file.
5. For any differing lines, it calls `find_diff_substrings` to identify the precise start column, old content, and new content, as required.
6. A vector of `UpdateObjects` is created with this data and a new timestamp.
7. Finally, the `last_mod_time` variable is updated to prevent immediate re-diffing.

2.2 Message Queues and Shared Memory Structure

- **Shared Memory (SHM):** We use `shm_open()` with `O_CREAT` to create or open the segment and `mmap()` to map it into the process's address space. Registration is lock-free, as described in *Design Decisions*.
- **Message Queues (MQ):** Each user creates their *own* queue with `mq_open(queue_name, O_CREAT | O_RDONLY | O_NONBLOCK)`.
  - **O_RDONLY:** The user only reads from their own queue.
  - **O_NONBLOCK:** The Listener Thread can poll the queue without blocking, though our implementation uses the blocking `mq_receive()` in a dedicated thread.
- **Broadcasting:** To broadcast, the Main Thread iterates the user list from SHM and calls `mq_open(other_queue_name, O_WRONLY)` for each peer, sends the message, and then calls `mq_close()`.

2.3 CRDT Merge Algorithm

The merge logic is triggered when the combined total of local and received operations reaches $N = 5$.

1. **Collect:** All `UpdateObjects` from `g_local_updates_buffer` and `g_received_updates_buffer` are combined into a single `all_ops` vector.
2. **Detect Conflicts:** The system uses a nested loop to compare every pair of operations in `all_ops`. A conflict is defined as two operations where `op_A.line_num == op_B.line_num` AND their column ranges overlap.
3. **Resolve Conflicts (LWW):** When a conflict is detected:
   - The timestamps are compared. The operation with the *older* (smaller) timestamp is marked as "discarded."
   - **Tiebreaker:** If timestamps are identical, the user IDs are compared as strings. The operation from the user with the *larger* (alphabetically greater) `user_id` is discarded.
4. **Apply:** A new vector of winning objects is created. These operations are then applied to the in-memory document state.
5. **Convergence:** Because all users receive all operations (eventually) and apply the exact same deterministic LWW rules, they are all mathematically guaranteed to converge to the identical final document state.

2.4 Thread Architecture and Communication

As shown in the diagram, the system uses two primary threads:

- **Main Thread:** This thread is the "conductor." It is responsible for:
  - Polling the local file (`stat()`).
  - Diffing changes.
  - Broadcasting local updates to peers.
  - Popping received updates from the SPSC queue.

  - Running the CRDT merge logic.
  - Updating the local file and the terminal display.

- **Listener Thread:** This thread is a simple "sentry." It has one job:
  - Block on `mq_receive()` waiting for an `MQ_UpdateObject` to arrive.
  - When an object arrives, push it onto the lock-free SPSC queue.
  - Loop back to `mq_receive()`.

- **Communication:** The **SPSC Queue** is the only bridge between them. This is the core of the lock-free design. The Main Thread is the *Single Consumer* and the Listener Thread is the *Single Producer*. This avoids any need for `std::mutex`, fulfilling the project's main constraint.

3 Design Decisions

3.1 Ensuring Lock-Free Operation

The entire project is built on two core lock-free mechanisms:

1. **Atomic Registration (SHM):** User registration in shared memory is a classic "race condition" problem. We solve this by using an `std::atomic<pid_t>` for each user slot. The `register_user()` function iterates the slots and uses `pid.compare_exchange_strong(expected, my_pid)`. This atomic hardware instruction guarantees that even if two users try to claim the same free slot (where `expected == 0`) at the same microsecond, only one will succeed.

2. **SPSC Queue (Threads):** Communication between the Main and Listener threads is another potential race condition. We solve this by implementing a lock-free Single-Producer, Single-Consumer queue. This data structure uses atomic read/write indices (`m_read_index`, `m_write_index`) and C++ memory ordering guarantees (`memory_order_release`, `memory_order_acqu` to ensure that data is safely passed from the producer (Listener) to the consumer (Main) without ever stalling or using a mutex.

3.2 File Monitoring: Polling vs. Kernel Events

- **Choice:** We implemented a simple polling mechanism using `stat()` every 500ms.

- **Rationale:** This approach was hinted at by the project specification and is simple, portable, and easy to implement.

- **Trade-off:** The alternative, `inotify` (a Linux kernel API), is far more efficient as it provides real-time events instead of polling. However, it is much more complex to implement and less portable. Given the project constraints, polling was the pragmatic and correct choice.

3.3 Data Serialization: `std::string` vs. `char[]`

- **Choice:** We used two separate structs: `UpdateObject` (with `std::string`) and `MQ_UpdateObject` (with `char[]`).

- **Rationale:** `std::string` is flexible and robust for C++ logic (like diffing). However, POSIX MQs require a fixed message size, which `std::string` does not provide.

- **Solution:** We perform serialization/deserialization at the "boundaries." When broadcasting, the Main Thread converts `UpdateObject` → `MQ_UpdateObject`. When receiving, the Main Thread converts `MQ_UpdateObject` → `UpdateObject`. This gives us the best of both worlds: C++ convenience internally and C-level compatibility for IPC.

4 Challenges and Solutions

4.1 Challenge 1: Self-Synchronization Feedback Loop

- **Problem:** The Main Thread would merge updates, write the new content to the local file, and this write would update the file's `st_mtime`. On the very next loop, the `stat()` check would detect this as a *new* change, diff its own merge, and broadcast it to everyone, causing an infinite loop.
- **Solution:** After a successful merge, the Main Thread calls `write_file()`. It then *immediately* calls `get_mod_time()` and updates its local `last_mod_time` variable. This "resets" the detector, making it aware that the newly written state is the correct source of truth.

4.2 Challenge 2: Linker Errors after Modularization

- **Problem:** After splitting the code into multiple `.cpp` and `.h` files, the linker produced hundreds of "multiple definition" errors for global constants like `SHM_REGISTRY_NAME`.
- **Solution:** This was a fundamental C++ ODR (One Definition Rule) violation. Variables defined in headers are redefined in every file that includes them. The fix was to:
  1. Declare the variable as `extern const char* SHM_REGISTRY_NAME;` in `common.h`.
  2. Provide the *single* definition, `const char* SHM_REGISTRY_NAME = "/synctext_registry";`, in `globals.cpp`.